

Yes — a practical, AI-revisable workflow system is very doable. Below is a clean, modular Python design (recommended over pure PowerShell for complex analytical/reporting work) that runs on Windows. It is deliberately structured so an AI can easily read, revise, or extend both the workflow definition and the individual tools.

Design Goals

- **Declarative workflow** (YAML or simple Python dict) that an AI can rewrite.
- **Tool registry:** each tool is a small, self-contained function/class with clear input/output schema.
- **Orchestrator** that executes steps in order, passes data between them, handles errors, and logs.
- **Easy AI revision:** tools live in one folder, workflow in one file; both have clear signatures and docstrings.
- Practical for analytical/reporting tasks (data load → clean → analyze → report → export).

Folder Structure

workflow_system/

```
├─ main.py          # Entry point + orchestrator
├─ workflow.yaml    # Declarative workflow (AI-friendly)
├─ tools/
│   ├─ __init__.py
│   ├─ base.py      # Base tool class + registry
│   ├─ load_data.py
│   ├─ clean_data.py
│   ├─ analyze.py
│   ├─ generate_report.py
│   └─ export.py
├─ config.yaml      # Optional settings
└─ logs/
```

```
from abc import ABC, abstractmethod
```

```
from typing import Any, Dict
```

```
import importlib

import pkgutil

from pathlib import Path
```

```
class BaseTool(ABC):
```

```
    """Every tool must inherit from this. AI can safely revise individual tools."""
```

```
    name: str = "base"
```

```
    description: str = ""
```

```
    required_inputs: list[str] = []
```

```
    produces: list[str] = []
```

```
    @abstractmethod
```

```
    def run(self, context: Dict[str, Any]) -> Dict[str, Any]:
```

```
        """Execute the tool. Receive shared context, return updates to context."""
```

```
        pass
```

```
# Simple registry
```

```
_TOOL_REGISTRY: Dict[str, BaseTool] = {}
```

```
def register_tool(tool: BaseTool):
```

```
    _TOOL_REGISTRY[tool.name] = tool
```

```
def get_tool(name: str) -> BaseTool:
```

```
    if name not in _TOOL_REGISTRY:
```

```
        raise ValueError(f"Tool '{name}' not found. Available:  
{list(_TOOL_REGISTRY.keys())}")
```

```
return _TOOL_REGISTRY[name]
```

```
def discover_tools(package_path: str = "tools"):
```

```
    """Auto-discover and register all tools in the tools package."""
```

```
    package = importlib.import_module(package_path)
```

```
    for _, module_name, _ in pkgutil.iter_modules(package.__path__):
```

```
        if module_name in ("base", "__init__"):
```

```
            continue
```

```
        mod = importlib.import_module(f"{package_path}.{module_name}")
```

```
        for attr in dir(mod):
```

```
            obj = getattr(mod, attr)
```

```
            if isinstance(obj, type) and issubclass(obj, BaseTool) and obj is not BaseTool:
```

```
                instance = obj()
```

```
                register_tool(instance)
```

2. Example Tools

tools/load_data.py

```
from tools.base import BaseTool, register_tool
```

```
from typing import Any, Dict
```

```
import pandas as pd
```

```
from pathlib import Path
```

```
class LoadDataTool(BaseTool):
```

```
    name = "load_data"
```

```
    description = "Load CSV or Excel file into a DataFrame"
```

```
    required_inputs = ["input_path"]
```

```
produces = ["df"]
```

```
def run(self, context: Dict[str, Any]) -> Dict[str, Any]:  
  
    path = Path(context["input_path"])  
  
    if path.suffix.lower() == ".csv":  
  
        df = pd.read_csv(path)  
  
    elif path.suffix.lower() in (".xlsx", ".xls"):  
  
        df = pd.read_excel(path)  
  
    else:  
  
        raise ValueError(f"Unsupported file type: {path.suffix}")  
  
    print(f"[load_data] Loaded {len(df)} rows from {path}")  
  
    return {"df": df}
```

tools/analyze.py

```
from tools.base import BaseTool
```

```
from typing import Any, Dict
```

```
import pandas as pd
```

```
class AnalyzeTool(BaseTool):
```

```
    name = "analyze"
```

```
    description = "Produce simple summary statistics and key metrics"
```

```
    required_inputs = ["df"]
```

```
    produces = ["summary", "metrics"]
```

```
def run(self, context: Dict[str, Any]) -> Dict[str, Any]:
```

```
    df = context["df"]
```

```
    summary = df.describe(include="all").to_dict()
```

```

metrics = {
    "row_count": len(df),
    "column_count": len(df.columns),
    "numeric_columns": df.select_dtypes(include="number").columns.tolist(),
}

print(f"[analyze] Generated summary for {metrics['row_count']} rows")

return {"summary": summary, "metrics": metrics}

```

tools/generate_report.py

```

from tools.base import BaseTool

```

```

from typing import Any, Dict

```

```

from datetime import datetime

```

```

class GenerateReportTool(BaseTool):

```

```

    name = "generate_report"

```

```

    description = "Create a plain-text / markdown report from analysis"

```

```

    required_inputs = ["metrics", "summary"]

```

```

    produces = ["report"]

```

```

    def run(self, context: Dict[str, Any]) -> Dict[str, Any]:

```

```

        metrics = context["metrics"]

```

```

        report = f"""# Analytical Report

```

```

Generated: {datetime.now().isoformat()}

```

```

## Key Metrics

```

```

- Rows: {metrics['row_count']}

```

```

- Columns: {metrics['column_count']}

```

- Numeric columns: {'', '.join(metrics['numeric_columns'])}}

Summary Statistics

(See full summary in context)

"""

print("[generate_report] Report created")

return {"report": report}

tools/export.py

from tools.base import BaseTool

from typing import Any, Dict

from pathlib import Path

class ExportTool(BaseTool):

name = "export"

description = "Write report and optional cleaned data to disk"

required_inputs = ["report"]

produces = []

def run(self, context: Dict[str, Any]) -> Dict[str, Any]:

out_dir = Path(context.get("output_dir", "output"))

out_dir.mkdir(exist_ok=True)

report_path = out_dir / "report.md"

report_path.write_text(context["report"], encoding="utf-8")

print(f"[export] Wrote report to {report_path}")

if "df" in context:

```
context["df"].to_csv(out_dir / "cleaned_data.csv", index=False)
```

```
return {}
```

3. Workflow Definition (workflow.yaml) – Easy for AI to revise

name: simple_analytical_report

description: Load → Clean → Analyze → Report → Export

steps:

- tool: load_data

params:

input_path: "data/sample.csv" # AI can change this

- tool: clean_data

- tool: analyze

- tool: generate_report

- tool: export

params:

output_dir: "output"

4. Orchestrator (main.py)

```
import yaml
```

```
from pathlib import Path
```

```
from typing import Any, Dict
```

```
from tools.base import discover_tools, get_tool
```

```

def load_workflow(path: str = "workflow.yaml") -> Dict[str, Any]:

    with open(path, encoding="utf-8") as f:

        return yaml.safe_load(f)


def run_workflow(workflow: Dict[str, Any], initial_context: Dict[str, Any] | None =
None):

    discover_tools() # auto-register everything in tools/

    context = initial_context or {}

    print(f"=== Starting workflow: {workflow.get('name')} ===")


    for i, step in enumerate(workflow["steps"], 1):

        tool_name = step["tool"]

        params = step.get("params", {})

        context.update(params) # inject step-level params


        tool = get_tool(tool_name)

        print(f"\n[{i}] Running tool: {tool_name} – {tool.description}")

        try:

            updates = tool.run(context)

            context.update(updates)

        except Exception as e:

            print(f"ERROR in {tool_name}: {e}")

            raise


    print("\n=== Workflow finished ===")

```



```
return context
```

```
if __name__ == "__main__":  
  
    workflow = load_workflow()  
  
    result = run_workflow(workflow)  
  
    # Optional: print final report  
  
    if "report" in result:  
  
        print("\n--- Final Report Preview ---")  
  
        print(result["report"][:500])
```

How an AI Revises This Easily

1. **Change the sequence or add steps** → edit workflow.yaml.
2. **Modify a tool's logic** → open the corresponding file in tools/ (clear class + run method).
3. **Add a new tool:**
 - Create tools/new_tool.py inheriting from BaseTool.
 - Set name, description, required_inputs, produces.
 - Implement run.
 - It is auto-discovered; just add the step to the YAML.
4. **Change parameters** → only the YAML (or inject via initial_context).

Running on Windows

```
pip install pandas pyyaml openpyxl
```

```
python main.py
```

Optional PowerShell Wrapper

If you prefer to launch from PowerShell:

```
# run_workflow.ps1
```

```
param(
```

```
    [string]$Workflow = "workflow.yaml"
```

```
)
```

```
python main.py # or pass --workflow $Workflow if you extend main.py
```

This pattern is practical, production-usable for analytical/reporting pipelines, and deliberately AI-friendly (small files, explicit interfaces, declarative control flow). You can extend it with retries, parallel steps, conditionals, or a simple GUI later if needed.